

# AeroNetB ASCM Video Presentation Script

| Field         | Value                                                                                                   |
|---------------|---------------------------------------------------------------------------------------------------------|
| Student Name  | Spyros Kokkoris                                                                                         |
| Student ID    | 100774175                                                                                               |
| Target Length | Approximately 10 minutes                                                                                |
| Live App      | <a href="https://aeronetb-ascm-dashboard.onrender.com">https://aeronetb-ascm-dashboard.onrender.com</a> |

## Opening

Hello, my name is Spyros Kokkoris, student ID 100774175. In this video I am presenting my 5CM506 Data Driven Systems implementation for the AeroNetB Aerospace Supply Chain Management scenario. The project continues the first-semester database design and turns it into a working cloud-hosted system with a web dashboard, backend APIs, PostgreSQL, and MongoDB.

## Project Purpose

AeroNetB manufactures aircraft components and depends on many specialised suppliers. The business problem is that supplier records, purchase orders, shipment updates, quality reports, certifications, and IoT logs are fragmented across different tools. My system brings these data areas together so the company can track orders, monitor supplier quality, inspect parts, lock certification records, and review audit trails from one dashboard.

## Architecture

The application uses a hybrid data architecture. PostgreSQL stores structured data such as suppliers, parts, supplier-part variants, purchase orders, shipments, users, roles, permissions, equipment, and audit logs. MongoDB stores flexible document data such as QC reports, certifications, IoT sensor readings, and compliance flags. This follows the polyglot persistence idea from the design document: stable relational data goes into a relational database, while variable JSON-style evidence goes into a document database.

## Deployment

The live application is deployed on Render at <https://aeronetb-ascm-dashboard.onrender.com>. The source code is public on GitHub at <https://github.com/Spyko2001/aeronetb-ascm-dashboard>. The Render health endpoint confirms that the live service is connected to both databases. During the demo, I show the endpoint `/api/health` returning `database_mode` as PostgreSQL plus MongoDB and `database_warning` as null.

## Authentication And RBAC

The application has five seeded demo users: procurement officer, quality inspector, supply chain manager, equipment engineer, and auditor. Each user logs in with an auth ID and the demo password demo123. Passwords are not stored as plain text in the runtime seed process; they are hashed using PBKDF2. After login, the server returns a signed session token. Every protected endpoint checks role permissions in the backend, not only in the interface.

## Dashboard Overview

I start the walkthrough as the supply chain manager. This overview shows KPI cards for

suppliers, open orders, delayed orders, failed QC reports, pending certifications, and IoT warnings. It also shows supplier performance and QC outcome charts. The alerts panel highlights delayed orders, QC failures, pending certifications, and sensor warnings. The search box and export controls support the general dashboard features required by the assignment.

## **Procurement Workflow**

Next I log in as the procurement officer. This role can create supplier records and purchase orders. The procurement page shows supplier contact and accreditation data, then order data with supplier, part lines, desired delivery dates, actual delivery dates, and current status. This demonstrates that the dashboard is not static; it uses API-backed data and allows permitted users to create operational records.

## **Shipment Workflow**

The shipment view shows tracking numbers, linked orders, suppliers, port of entry, ETA, latest event location, and status. Shipment events are stored as separate records so multiple checkpoints and condition updates can be recorded over the shipment lifecycle. This supports the scenario requirement for real-time visibility across orders and shipments.

## **Quality Workflow**

Then I log in as the quality inspector. This role can create QC reports and certifications. QC reports are stored in MongoDB because their result data changes depending on the inspection type, for example visual, dimensional, NDT, or environmental testing. If a report is repeated for the same order, part, supplier, and report type, the system creates a new version instead of overwriting the old report. This preserves version history.

## **Certification Immutability**

The quality inspector can also create certifications and approve them. When a certification is approved, the system sets `is_immutable` to true and records approval metadata. Any later update attempt returns an error. This demonstrates the required immutability rule for approved certifications and digital signatures.

## **IoT Workflow**

Next I log in as the equipment engineer. This role sees equipment status and IoT telemetry. The IoT readings include values such as temperature, vibration, pressure, and GPS position. The system calculates warning or critical status from the latest readings, which supports the dashboard requirement for equipment monitoring and predictive maintenance alerts.

## **Auditor Workflow**

Finally I log in as the auditor. The auditor can review certification and compliance records but cannot modify core operational records. The auditor can raise a compliance flag if a record needs attention. This demonstrates read-only compliance access with a controlled exception for non-compliance flagging.

## **Audit Trail**

The audit log records important actions with employee ID, action type, entity type, entity

ID, timestamp, outcome, and details. This is important because the scenario requires every access or modification to be attributable to the correct empld. I show audit records after logging in and performing actions across roles.

## **Source Code And Evidence**

The source code is organised into backend and frontend folders. The backend contains the API server, database layer, security logic, seed data, and domain aggregation code. The SQL scripts folder contains PostgreSQL DDL and DML evidence. The Mongo scripts folder contains collection setup, validators, and indexes. The documents include the database design document, implementation logbook, screenshots, and this presentation script.

## **Closing**

To conclude, the system demonstrates a working relational database, working MongoDB collections, API endpoints, role-based login, dashboard interaction, RBAC enforcement, audit logging, QC versioning, certification immutability, and cloud deployment on Render. This completes the second-semester implementation as a continuation of the first-semester design.